

COMPUTATIONAL PHYSICS | MARCH 01 2026

Introduction to supervised machine learning for physics students ✓

Jan Tobochnik ✉  ; Harvey Gould 



Am. J. Phys. 94, 224–229 (2026)

<https://doi.org/10.1119/5.0285290>



Articles You May Be Interested In

Machine learning based classification of vector field configurations

AIP Advances (February 2024)

Reference Correlation of the Viscosity of *n*-Heptane from the Triple Point to 600 K and up to 248 MPa

J. Phys. Chem. Ref. Data (June 2014)

Stabilization of microcrystal λ - Ti_3O_5 at room temperature by aluminum-ion doping

Appl. Phys. Lett. (November 2017)

The Computational Physics Section publishes articles that help students and instructors learn about the computational tools used in contemporary research. Interested authors are encouraged to send a proposal to the editors of the Section, Jan Tobochnik (jant@kzoo.edu) or Harvey Gould (hgould@clarku.edu). Summarize the physics and the algorithm you wish to discuss and how the material would be accessible to advanced undergraduates or beginning graduate students.

Introduction to supervised machine learning for physics students

Jan Tobochnik^{1,a)} and Harvey Gould^{2,b)}

¹*Department of Physics, Brown University, Providence, Rhode Island 02912 and Department of Physics, Kalamazoo College, Kalamazoo, Michigan 49006*

²*Department of Physics, Boston University, Boston, Massachusetts 02215 and Department of Physics, Clark University, Worcester, Massachusetts 01610*

(Received 13 June 2025; accepted 23 January 2026)

We discuss a simple supervised machine learning algorithm using logistic regression to classify phases in the Ising model of magnetization and the trajectories of a particle subject to either a spring or a gravitational force. Several pedagogical problems are provided as a means to introduce students to the basic ideas of machine learning. © 2026 Published under an exclusive license by American Association of Physics Teachers.

<https://doi.org/10.1119/5.0285290>

I. INTRODUCTION

Machine learning is being used in many contexts, including the natural sciences,^{1–7} medicine,⁸ and the social sciences.^{9,10} For example, machine learning has been used in physics to detect signals in noisy data¹¹ and to find specific signals in large datasets.¹² Machine learning has been used in astronomy to detect exoplanets and other astrophysical objects.¹³ Some educational resources are available in Refs. 14–17.

The three broad types of machine learning are supervised learning, unsupervised learning, and reinforcement learning.^{4,15,18–21} Supervised learning is typically used for fitting and classifying data. When used for classification, a program is trained on data for which the classification is known. Unsupervised learning has no training and is used to search for patterns in the data. Reinforcement learning is used when an agent such as a robot takes in data from a dynamical environment, then takes some actions to reach a goal, and then uses the resultant feedback on how well the goal was reached to modify its future actions.

In this paper, we discuss classification using supervised learning. The paper is organized as follows. We provide a conceptual introduction on the nature of supervised machine learning in Sec. II and discuss an example for which classification is not straightforward. In Sec. III, we describe a machine learning algorithm that determines the phases (paramagnetic or ferromagnetic) of spin configurations generated by simulations of the Ising model of magnetism. We also discuss how well the algorithm works. In Sec. IV, we apply the same algorithm to distinguish data from two mechanical systems, either a two-dimensional spring or planetary motion. We conclude in Sec. V with some general comments and some suggested problems for students, which can be used in the classroom or as homework assignments.

II. WHAT IS SUPERVISED MACHINE LEARNING?

We first discuss a non-physics problem to illustrate some of the basic ideas of supervised learning. Suppose that we wish to design a machine learning algorithm that distinguishes spam from not spam in our emails. Our first task is to choose some features that distinguish spam from not spam emails. Some obvious features in spam emails are keywords such as “free,” “prize,” and “locked.” How would the use of supervised machine learning treat this problem? We would first begin with a set of training data consisting of many emails, each labeled as either spam or not spam. We then could define three features of the email as how often each of the three keywords were found in an email, and use these features to predict whether the email is spam or not.

How much weight should be given to each feature that we have identified? Are there any hidden correlations between these three features? The goal of machine learning is to determine the weight of each feature by starting with arbitrary weights, computing the predictions using the weights, and then modifying the weights based on comparing the spam or not spam predictions with the actual training data. We repeat this modification of the weights for many iterations. The simplest algorithm for determining the optimum weights is logistic regression, which is discussed in Sec. III. Eventually, the change in the weights will be sufficiently small so that further iterations of this procedure are not useful.

Once the weights are determined, we use them to predict the nature of the emails on a set of test data that is distinct from the training data. If the test results are satisfactory, we can then use our algorithm for email not in the training or testing dataset. If not, we must consider the possible sources for the lack of success. We might need more training data or choose a different set of features. Finally, logistic regression

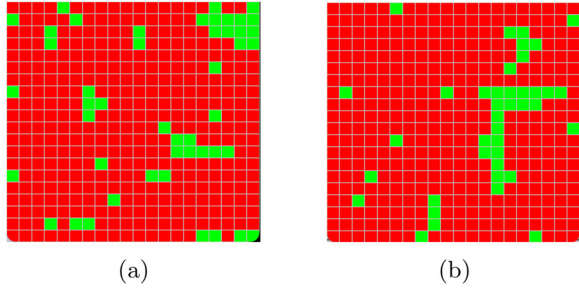


Fig. 1. Typical Ising model configurations for (a) $T^* = 2.2$ and for (b) $T^* = 2.3$. Red cells indicate spin up and green cells indicate spin down. Periodic boundary conditions were used in the Monte Carlo algorithm generating the configurations. Because the lattice is small and the temperature is close to T_c^* , the magnetization is large.

might not be sufficient for this problem, so that more powerful machine learning algorithms are needed, which is the case for identifying spam email. For detecting spam, a useful algorithm should be continuously updated because spam emails are becoming more and more sophisticated.

III. SUPERVISED LEARNING ALGORITHM FOR CLASSIFYING THE PHASE OF THE ISING MODEL

Consider the Ising model on a square lattice. In zero magnetic field, the energy per spin of a configuration is given by

$$E = -\frac{J}{N} \sum_{\langle ij \rangle} s_i s_j, \quad (1)$$

where $N = L^2$ is the number of spins on an $L \times L$ lattice, $s_i = \pm 1$, the sum is over the nearest-neighbor pairs of spins (each spin has four nearest neighbors on a square lattice) and the exchange energy $J > 0$ for ferromagnetic interactions. In the sum, we are careful to avoid double counting. For example, we include only one of the two terms $s_1 s_2$ and $s_2 s_1$. The magnetization of a configuration is given by

$$M = \frac{1}{N} \sum_i s_i. \quad (2)$$

We define the dimensionless temperature $T^* = k_B T / J$, where T is the absolute temperature and k_B is the Boltzmann constant. The critical temperature between a ferromagnetic and a paramagnetic state for the infinite lattice is $T_c^* = 2 / \ln(1 + \sqrt{2}) \approx 2.269$.

Figure 1 shows typical configurations for $T^* > T_c^*$ and for $T^* < T_c^*$ for $L = 20$. As can be seen, the configurations appear similar, and thus it is a challenge for an algorithm to determine which configuration came from a simulation slightly above T_c^* and which came from a simulation slightly below T_c^* .

We begin by describing the training algorithm.* All supervised machine learning algorithms have a list of features and a list of classes which are used to characterize the input data. The first step is to obtain the input data by generating a random set of N_{config} independent spin configurations. To do so, we begin with all spins up, do a Metropolis Monte Carlo (MC) simulation²⁴ at a given temperature chosen at random

*The Ising model has been discussed using more sophisticated machine learning algorithms in Refs. 22 and 23.

in the desired temperature range near T_c^* , and then use the spin configuration generated after $N_{\text{iter}} = 1000$ Monte Carlo steps per spin. Because we know the temperature used in the MC simulation, we know the corresponding phase of each configuration. Each new configuration is started with all spins up. In this example, the classification is straightforward and is the phase associated with each configuration. For each training configuration i , we define the quantity $\phi_i = 1$ for the ordered ferromagnetic phase and $\phi_i = 0$ for the disordered paramagnetic phase.

The ease of computing the features and the relevance of the features to the classification are important considerations when deciding what features to use. For each configuration, we compute the energy per spin and the magnetization using Eqs. (1) and (2), which we choose to be the features. These two features are relevant to the classification problem because the energy and the magnetization behave differently above and below T_c .

The next step is to determine the weights that yield the best predictions for the phases of the training configurations. We use logistic regression to determine the weights. We define a vector

$$\mathbf{x}_i = [1, M_i, E_i] \quad (3)$$

for each training configuration i . We also define the initial vector of weights $\mathbf{w} = [0, 0, 0]$, and compute the phase predictor p_i as

$$p_i = 1 / (1 + e^{-d_i}), \quad (4)$$

where

$$d_i = \mathbf{x}_i \cdot \mathbf{w}. \quad (5)$$

We also define a prediction for the disordered phase as $p_i < 1/2$ and for the ordered phase as $p_i > 1/2$. Thus, according to Eq. (4), the dividing line between the two phases occurs for $d_i = 0$, where $p_i = 1/2$. Because there is no reason why our finite collection of spin configurations will be centered about $d_i = 0$ for a collection of M and E values, we include a bias element equal to unity as the first element in $\mathbf{x}_i$ in addition to M and E as shown in Eq. (3).

To adjust the weights, we need a measure of how well the predictors $\{p_i\}$ compare to the actual set of values $\{\phi_i\}$. In analogy to the Shannon entropy $S = -\sum_e P_e \log P_e$, where P_e is the probability of an event e occurring, we define the mean cross-entropy loss function as

$$\mathcal{L} = -\frac{1}{N_{\text{config}}} \sum_{i=1}^{N_{\text{config}}} \phi_i \log p_i + (1 - \phi_i) \log (1 - p_i), \quad (6)$$

where i labels the configuration number and $\phi_i = 0$ or 1 . This definition of entropy loss is reasonable because if $\phi_i = 1$, then the i th term contributes $-(1/N_{\text{config}}) \log p_i$, which is small if p_i is close to unity (recall that $p_i > 1/2$ corresponds to the prediction $\phi_i = 1$). Similarly, if $\phi_i = 0$, then the i th term contributes $-(1/N_{\text{config}}) \log (1 - p_i)$, which is small if $1 - p_i$ is close to unity ($p_i < 1/2$ corresponds to the prediction $\phi_i = 0$).

The gradient with respect to $\mathbf{w}$ tells us how to change the weights. To determine this gradient, note that

$$\begin{aligned}
\nabla_{\mathbf{w}} p_i &= \nabla_{\mathbf{w}} (1 + e^{-\mathbf{x}_i \cdot \mathbf{w}})^{-1} \\
&= -(1 + e^{-\mathbf{x}_i \cdot \mathbf{w}})^{-2} (-\mathbf{x}_i) e^{-\mathbf{x}_i \cdot \mathbf{w}} = p_i^2 \mathbf{x}_i \left(\frac{1}{p_i} - 1 \right) \\
&= (p_i - p_i^2) \mathbf{x}_i,
\end{aligned} \tag{7}$$

where we have used Eqs. (4) and (5). We use Eq. (7) and some algebra to write the gradient vector as

$$\mathbf{g} = \nabla_{\mathbf{w}} \mathcal{L} = \frac{1}{N_{\text{config}}} \sum_{i=1}^{N_{\text{config}}} (p_i - \phi_i) \mathbf{x}_i. \tag{8}$$

Recall that $0 < p_i < 1$ is used to predict the phase and $\phi_i = 0$ or 1 is the known phase associated with the i th configuration. The weights are now adjusted as

$$\mathbf{w}(k+1) = \mathbf{w}(k) - \eta \mathbf{g}, \tag{9}$$

where k is the iteration number and η is the learning rate. At each iteration, the weights are changed to improve the predictions for the training data.

We update the weights for N_{iter} iterations. Figure 2 shows how the magnitudes of the gradients decrease with iteration number. Figure 3 shows how the weights tend to limiting value as the iteration number k increases. If we use the weights at $k = 1000$, then for $\mathbf{x}_i = (1, 1, -2)$ corresponding to a perfectly ordered state, we find from Eqs. (4) and (5) that $p_i \approx 0.91 > 0.5$, which corresponds to the correct prediction of an ordered phase. If we use $\mathbf{x}_i = (1, 0, 0)$ corresponding to a very disordered state, then $p_i \approx 0.012 < 0.5$, which corresponds to the correct prediction for a disordered phase. This procedure completes the training part of the machine learning algorithm.

To test how well this algorithm works, we generate configurations using the Metropolis Monte Carlo algorithm at temperatures in the same range as the training data, $1.5 \leq T^* \leq 4.5$ with $L = 20$, $N_{\text{config}} = 500$, $\eta = 0.1$, and $N_{\text{iter}} = 1000$.

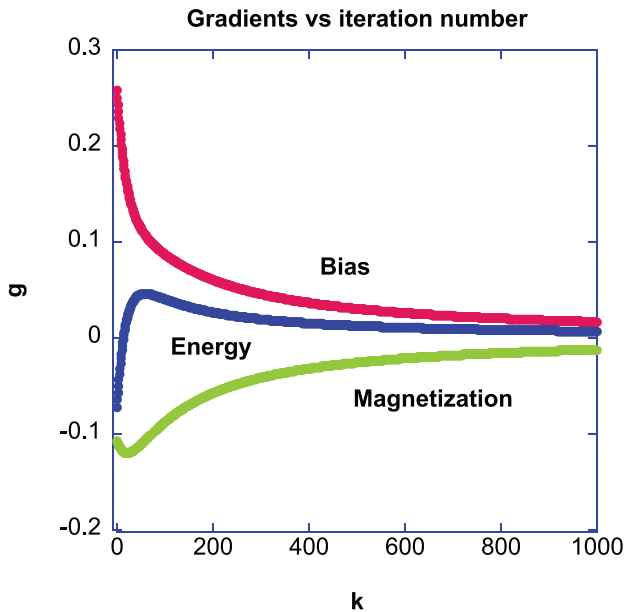


Fig. 2. The dependence of the elements of the gradient for the weights on the iteration number k for the Ising model for $L = 20$, $N_{\text{config}} = 500$, and $\eta = 0.1$.

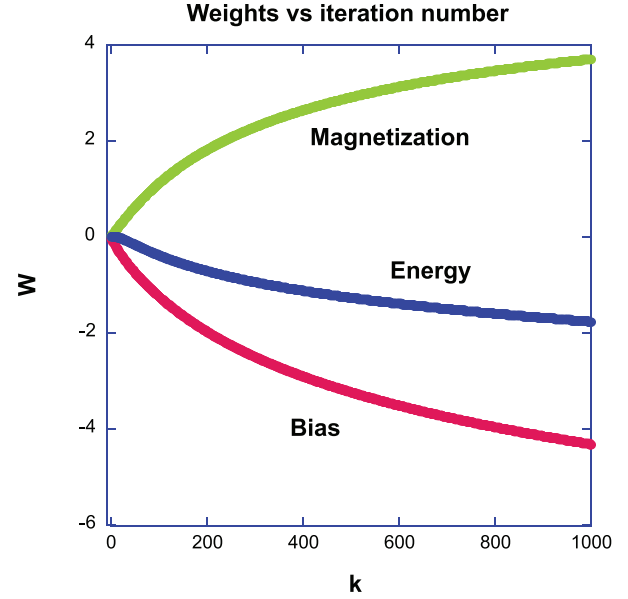


Fig. 3. The dependence of the weights on the iteration number k for the Ising model for $L = 20$, $N_{\text{config}} = 500$, and $\eta = 0.1$.

We use the weight vector $\mathbf{w}$ in Eq. (9) after N_{iter} iterations to compute our prediction using Eq. (4) and then determine if our prediction is correct. Because we know T_c , we know to which phase each test configuration belongs. By using test configurations in the same temperature range as our training configurations, we found the probability of success to be $\approx 97\%$, with a *confidence interval* or uncertainty of less than 1%. Repeating the process without the bias in $\mathbf{x}_i$ (i.e., only two elements, M_i and E_i , in $\mathbf{x}_i$) gives a success probability slightly lower and equal to $\approx 95\%$. If we use training data with a different range $2.0 \leq T^* \leq 4.5$, there is only a 74% success probability without the bias, but with the bias the success probability is still $\approx 97\%$. Thus, the use of the bias term in the vector $\mathbf{x}_i$ is very important.

The results are insensitive to N_{iter} and N_{config} . Only when N_{iter} is less than about 50 does the success rate start to decrease. A plot of the success rate versus N_{config} is called the *learning curve*, which here starts to decline only for N_{config} less than about 10. This dependence suggests that at least one of the features changes drastically at T_c , which is known to be true, and that as long as there are some configurations in the training set which are above and below T_c , the classification will be highly successful.

For our example, the success rate is very good. In some cases, there are features which dominate the training data and thus overfitting to the training data can occur such that the success rate is poor when the algorithm is applied to test data. *Regularization* is a technique for reducing the weights of certain features by penalizing those weights in evaluating the loss function during training.²⁵ In the Ising classification model, we only used two features. In the first problem in Sec. VI, you are asked to explore the effect of using only one feature to see if regularization is needed.

IV. CLASSIFYING MECHANICAL SYSTEMS

We now consider a different kind of classification problem. Imagine that the available data for a particle moving in the x - y plane is only the average of its x -coordinate squared, $\langle x^2 \rangle$, and the observation time T_{obs} over which the average is

computed. These two quantities will be the features used in the vector $\mathbf{x}$. We assume these data come from two completely different systems, namely, a particle moving in the x - y plane connected by a spring and the motion of a planet subject to the gravitational pull of the sun. Figure 4 shows two typical trajectories, one from a simulation of the motion of the particle connected to a spring and the other from a simulation of the motion of a planet. As we can see, the trajectories look similar.

We choose dimensionless units so that the time for one oscillation of the particle connected to a spring is the same order of magnitude as the time in years of a planet at one astronomical unit (AU), and the position of the particle in units so that its position is of the same order of magnitude as the position of the planet in AU. In these units, the quantity $GM_{\odot} = 4\pi^2$, where G is Newton's universal gravitational constant and $M_{\odot}$ is the mass of the sun. We choose the spring constant $k = 4\pi^2$ and the mass $m = 1$ of the particle attached to the spring. In these units, the initial position $[1, 0]$ and velocity $[0, 2\pi]$ produce circular trajectories with a period of one for both systems.

The equation of motion for the spring is

$$\frac{d^2x}{dt^2} = -kx, \quad (10a)$$

$$\frac{d^2y}{dt^2} = -ky. \quad (10b)$$

For the planet, we have

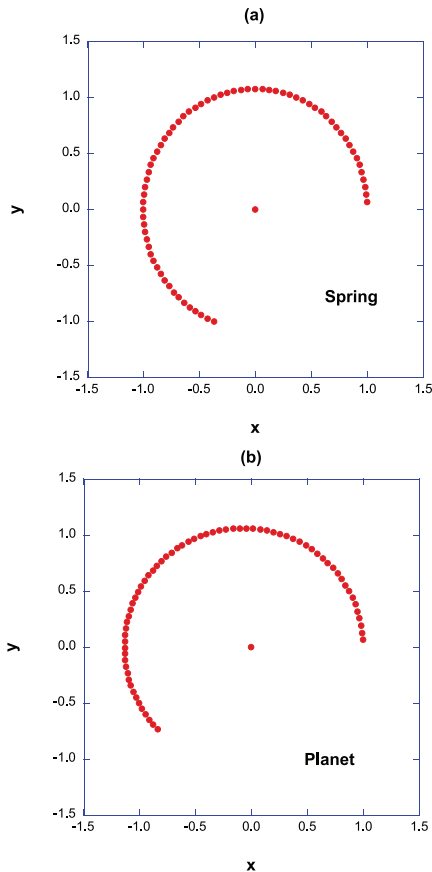


Fig. 4. Simulated trajectories for a (a) particle connected to a spring and for a (b) planet, both for the observation time $T_{\text{obs}} = 0.7$.

$$\frac{d^2x}{dt^2} = -\frac{GM_{\odot}x}{r^3}, \quad (11a)$$

$$\frac{d^2y}{dt^2} = -\frac{GM_{\odot}y}{r^3}, \quad (11b)$$

where $r^2 = x^2 + y^2$. We choose an initial position $\mathbf{r}_0 = [1, 0]$ and initial velocity $\mathbf{v}_0 = [0, 2\pi + \text{rnd}]$, where rnd is a random number between 0 and 1 for both systems. This choice ensures that the planet's orbit is elliptical. For both systems we consider random observation times $0 \leq T_{\text{obs}} \leq 1$. For the training data, we randomly choose each of the two systems with equal probability. Alternatively, we could simply choose half the input data from one system and half from the other. We used the velocity-Verlet algorithm to solve Eqs. (10) and (11) with $\Delta t = 0.01$ to compute $\langle x^2 \rangle$ for a time T_{obs} , but any other reasonable numerical algorithm will work.

Can the machine learning algorithm we discussed in Sec. III be used to determine which system gives particular values of $\langle x^2 \rangle$ for a specific value of T_{obs} ? Although this problem is artificial, it is interesting to see how well machine learning works. Because the physics of these two mechanical systems is very different than the Ising model, we need to choose different features and obtain the appropriate weights. We use the same supervised machine learning algorithm as for the Ising model with $N_{\text{config}} = 500$, $\eta = 0.1$, $N_{\text{iter}} = 1000$, and an initial bias equal to unity, and obtain $\approx 75\%$ success probability. Independent runs give slightly different values between $\approx 72\%$ and $\approx 79\%$, because the weights vary depending on the training data. This variation and the lower success rate for this classification problem compared to that for the Ising model indicates that this problem is more difficult for machine learning. If we set the bias equal to zero, the success probability decreases to only 56%–60%, which is only slightly better than a random guess.

V. SUMMARY

We have discussed one of the simplest supervised machine learning algorithms, which is particularly useful for problems for which the classification is known. One of the advantages of machine learning is that the same algorithm can be used for many different problems. The part of the machine learning algorithm that varies is the input data for training and which features from these data are chosen to generate the weights in the machine learning algorithm. The multipurpose algorithm we have discussed also illustrates one of the limitations of machine learning, which is that even though the algorithm can be good at yielding answers, it does not usually provide much insight into the physics of the systems being studied. The physics is mainly contained in the choice of features. In some problems, we might be able to discern unexpected correlations between quantities if adding an extra feature changes the success rate.

Our discussion of machine learning was straightforward because it was easy to find features that are useful in determining the classification. Frequently, the features to use are not obvious and classification is very difficult. For the two-dimensional Ising model, convolutional neural networks were used to successfully predict the temperature dependence of the magnetization, energy, and correlation length.^{22,23} Supervised machine learning and convolutional neural networks were used to classify whether or not a large earthquake occurs given an image of the stress of the sites as

a function of time before a large earthquake occurred in the Olami–Feder–Christensen model.²⁶ Although the training data included when large earthquakes occurred, the prediction of large earthquakes was possible only for a small subset of the test data.

More generally, the most difficult part of using machine learning is to develop algorithms beyond the simple one we have used for classifying the data, and it is common to use convolutional neural networks and other “black boxes” to do so.¹⁷ A more comprehensive and sophisticated introduction to machine learning in physics can be found, for example, in Ref. 21.

VI. SUGGESTED PROBLEMS

- Write a program to reproduce the results of the Ising model classification problem from Sec. III or use the program available in the [supplementary material](#). Make sure you structure your program so that the generation of the Ising model configurations and the extraction of E and M is separate from the rest of the program so that your program can be used as a template for other classification problems.
 - Determine which of the input parameters N_{config} , N_{iter} , η , and L have the biggest effect on the success probability.
 - Vary the range of temperatures used for the training data and that used for the test data. Is the success maximized when the two ranges are the same? What is the effect of smaller ranges versus larger ranges? What is the effect if the range of the training data is smaller than the test data, and vice versa? Use ranges where T_c is approximately in the middle of the temperature range. What happens if you do not do so?
 - Use only one feature, either the energy or magnetization. Which feature does better? Discuss the physics you can learn from your results.
 - Thus far, we have generated the configurations starting with all spins up, so that the configurations used for training and testing will have a positive magnetization unless the temperature is much greater than T_c , when the magnetization will be randomly positive or negative but near zero. What changes if you start with all spins down? Focus on the plots of the gradients and the weights. What happens if the initial spins are chosen randomly to be up or down? In all these cases, what happens if you use $|M_i|$ instead of M_i as a feature?
- Write a program to reproduce the results of the mechanics classification problem from Sec. IV. If you have already written a program for Problem 1, you will need only to make changes for generating the input data. Otherwise, make sure you structure your program so that the generation of the trajectories and the extraction of $\langle x^2 \rangle$ are separate from the rest of the program. Explore relevant issues from Problem 1. One check on your program is to set the initial position to $[1, 0]$ and velocity to $[0, 2\pi]$ and use random observation times T_{obs} . Your success probability should be 50%. Why? You can also test your program with different initial conditions, which include hyperbolic planetary orbits. Also, try other features to use. Discuss what physics you can learn from your results.

- In site percolation, the sites of a lattice are randomly occupied with probability p .²⁷
 - Consider a $L \times L$ square lattice, choose a value of $p = p^*$, and determine if we can use machine learning to determine whether a given configuration corresponds to one generated for $p < p^*$ or for $p > p^*$. Use features such as N_{edge} , the number of edges of the lattice that have an occupied site, and N_{occup} , the number of occupied neighbors of an occupied site. First consider $p^* = 0.4$ and a training and a testing range $0.3 < p < 0.5$. Also consider $p^* = 0.7$ and a training and testing range of $0.6 < p < 0.8$. Use $L = 40$, and the same values for the machine learning algorithm parameters N_{config} , η , and N_{iter} used in the discussion of the Ising model and the mechanics problems. What success rate do you obtain? Repeat for $L = 100$ and determine if the success rate changes.
 - Percolation is of particular interest in statistical physics because it exhibits a geometrical phase transition with properties similar to continuous thermal phase transitions.²⁷ The defining property of percolation is whether a spanning cluster of connected occupied sites exists. In the limit $L \rightarrow \infty$, there is a sharp transition at $p = p_c$, where p_c is the percolation threshold. For $p \geq p_c$, there is a spanning cluster and for $p < p_c$ there is no spanning cluster. For a square lattice, $p_c \approx 0.5927$. Because it is difficult to determine computationally whether or not a spanning cluster exists for a particular configuration,²⁸ it is interesting to determine if our choice of simple local features of the configurations in part (a) can be used to successfully predict the value of p_c . Choose values of p^* close to the known value of p_c and determine the success rate. What can you learn from this problem? It turns out that the percolation transition is a *global* property and thus we expect that using *local* features such as those suggested in part (a) is not effective at detecting the phase transition.²⁹
- Create your own machine learning problem. Use the machine learning algorithm from Problems 1 or 2 to write a machine learning program to do the prediction. Assume the classification is known for the training and test data.

SUPPLEMENTARY MATERIAL

Please click on [this link](#) to access the [supplementary material](#) contains a Python program for the Ising model classification problem. Print readers can see the [supplementary material](#) at <https://doi.org/10.60893/figshare.ajp.c.8258992>.

ACKNOWLEDGMENTS

The authors thank Panagiota Banti for providing the Python program for the Ising model classification problem.

AUTHOR DECLARATIONS

Conflict of Interest

The authors have no conflicts to disclose.

Author Contributions

Both authors contributed equally to writing the manuscript. Tobochnik wrote the computer programs in Java and collected the data used for the figures.

^{a)}Electronic mail: jan_tobochnik@brown.edu, ORCID: 0000-0002-8715-0014.

^{b)}ORCID: 0000-0002-2868-4577.

¹Joe G. Greener, Shaun M. Kandathil, Lewis Moffat, and David T. Jones, “A guide to machine learning for biologists,” *Nat. Rev. Mol. Cell Biol.* **23**(1), 40–55 (2022).

²Nongnuch Artrith, Keith T. Butler, François-Xavier Coudert, Seungwu Han, Olexandr Isayev, Anubhav Jain, and Aron Walsh, “Best practices in machine learning for chemistry,” *Nat. Chem.* **13**, 505–508 (2021).

³George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang, “Physics-informed machine learning,” *Nat. Rev. Phys.* **3**(6), 422–440 (2021).

⁴Giuseppe Carleo, Ignacio Cirac, Kyle Cranmer, Laurent Daudet, Maria Schuld, Naftali Tishby, Leslie Vogt-Maranto, and Lenka Zdeborová, “Machine learning and the physical sciences,” *Rev. Mod. Phys.* **91**(4), 045002 (2019).

⁵Y. Xie, X. Li, and Z. Min, “Comparison of machine learning models for rock UCS prediction using measurement while drilling data,” *Sci. Rep.* **15**(1), 8434 (2025).

⁶Guangping Li, Zujia Lu, Junzhi Wang, and Zhao Wang, “Machine learning in stellar astronomy: Progress up to 2024,” *arXiv:2502.15300* (2025).

⁷Xiangyu Zhang and Gregory M. Green, “Three-dimensional maps of the interstellar dust extinction curve within the Milky Way galaxy,” *Science* **387**(6739), 1209–1214 (2025). The researchers trained a machine learning model on 130×10^6 high quality stars and then used the model to reconstruct a high-resolution dust map for the rest of the 130×10^6 stars.

⁸Alvin Rajkomar, Jeffrey Dean, and Isaac Kohane, “Machine learning in medicine,” *N. Engl. J. Med.* **380**(14), 1347–1358 (2019).

⁹Margaret E. Roberts and Brandon M. Stewart, “Machine learning for social science: An agnostic approach,” *Ann. Rev. Pol. Sci.* **24**(1), 395–419 (2021).

¹⁰Nataša Kovač, Kruna Ratković, Hojjatollah Farahani, and Peter Watson, “A practical applications guide to machine learning regression models in psychology with Python,” *Methods Psychol.* **11**, 100156 (2024).

¹¹Zheng-Meng Zhai, Mohammadamin Moradi, Ling-Wei Kong, and Ying-Cheng Lai, “Detecting weak physical signal from noise: A machine-learning approach with applications to magnetic-anomaly-guided navigation,” *Phys. Rev. Appl.* **19**(3), 034030 (2023).

¹²Jelena Čulić Gambiroxa, Toni Mastelić, Ivana Nizetić Kosović, and Mario Čagalj, “Lost in data: Recognizing type of time series sensor data using signal pattern classification,” *Int. J. Data Sci. Anal.* **20**, 397–408 (2023).

¹³Jason Terry, Cassandra Hall, Sean Abreau, and Sergei Gleyzer, “Locating hidden exoplanets in ALMA data using machine learning,” *Astrophys. J.* **941**(2), 192 (2022).

¹⁴Viviana Acquaviva, “Teaching machine learning for the physical sciences: A summary of lessons learned and challenges,” in *Proceedings of the Second Teaching Machine Learning and Artificial Intelligence Workshop*, edited by M. Katherine, Peter Steinbach, and Oliver Guhr (PMLR, 2022), Vol. 170, pp. 35–39.

¹⁵Viviana Acquaviva, *Machine Learning for Physics and Astronomy* (Princeton, 2023).

¹⁶Karan Shah, Julie Butler, Alexis V. Knaub, Anil Zenginoğlu, William Ratcliff, and Mohammad Soltanieh-ha, “Data science education in undergraduate physics: Lessons learned from a community of practice,” *Am. J. Phys.* **92**(9), 655–662 (2024).

¹⁷See, for example, C. Scholz and S. Scholz, “Exploring complex pattern formation with convolutional neural networks,” *Am. J. Phys.* **90**, 141–151 (2022).

¹⁸A. Decelle, “An introduction to machine learning: A perspective from statistical physics,” *Physica A* **631**, 128154 (2023).

¹⁹Djenabou Bayo, Burak Cıvıtcioğlu, Joseph J. Webb, Andreas Honecker, and Rudolf A. Römer, “Machine learning of phases and structures for model systems in physics,” *J. Phys. Soc. Jpn.* **94**(3), 031002 (2025).

²⁰F. A. Rodrigues, “Machine learning in physics: A short guide,” *Europhys. Lett.* **144**(2), 22001–22007 (2023).

²¹P. Mehta, M. Bukov, C.-H. Wang, A. G. R. Day, C. Richardson, C. K. Fisher, and D. J. Schwab, “A high-bias, low-variance introduction to Machine Learning for physicists,” *Phys. Rep.* **810**, 1–124 (2019).

²²F. D’Angelo and L. Böttcher, “Learning the Ising model with generative neural network,” *Phys. Rev. Res.* **2**, 023266 (2020).

²³Burak Cıvıtcioğlu, Rudolf A. Römer, and Andreas Honecker, “Machine learning the square-lattice Ising model,” *J. Phys.: Conf. Ser.* **2207**(1), 012058 (2022).

²⁴See, for example, G. Tartero and W. Krauth, “Concepts in Monte Carlo sampling,” *Am. J. Phys.* **92**, 65–77 (2024) and W. Wang, “An introduction to the Markov chain Monte Carlo method,” *Am. J. Phys.* **90**, 921–934 (2022).

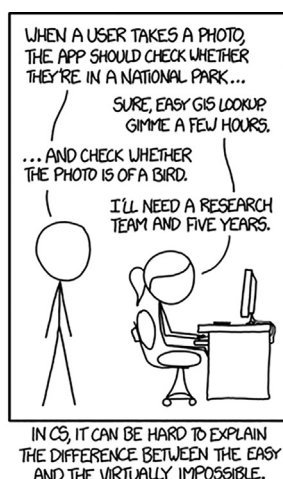
²⁵Shai Shalev-Shwartz and Shai Ben-David, “Chapter 13 discusses regularization,” *Understanding Machine Learning: From Theory to Algorithms* (Cambridge U. P., 2014).

²⁶Chon-Kit Pun, S. Matin, W. Klein, and H. Gould, “Prediction in a driven-dissipative system displaying a continuous phase transition using machine learning,” *Phys. Rev. E* **101**, 022102 (2020).

²⁷A. Aharony and D. Stauffer, *Introduction to Percolation Theory* (Taylor & Francis, 2003).

²⁸M. E. J. Newman and R. M. Ziff, “Fast Monte Carlo algorithm for site or bond percolation,” *Phys. Rev. E* **64**(1), 016706 (2001).

²⁹D. Bayo, A. Honecker, and R. A. Römer, “The percolating cluster is invisible to image recognition with deep learning,” *New J. Phys.* **25**(11), 113041 (2023).



In the 60s, Marvin Minsky assigned a couple of undergrads to spend the summer programming a computer to use a camera to identify objects in a scene. He figured they'd have the problem solved by the end of the summer. Half a century later, we're still working on it. (Source: <https://xkcd.com/1425>)